Assignment 2: X-ray Powder Diffraction (XRD) Diffractometer Simulation

Course: Nanotechnology & Quantum Physics | **Topic:** instrument simulation, Bragg-Brentano geometry, structure factors, crystallite size

Students: Mohamed Oboda (2111011090) | Mert SOLMAZ (2311011039)

Status: draft. The graded questions are not yet in this document because the assignment brief has not been received; everything below describes the simulator itself.

Item	What it is
Simulator	Cu K-alpha tube, divergence and Soller slits, flat sample on a theta/2theta goniometer, receiving slit, optional monochromator, detector with Poisson counting statistics
Phase database	Si, Al, Cu, alpha-Fe, NaCl, MgO, ZnO, TiO2 rutile and anatase, with the source of every cell constant in `data/SOURCES.md`
Physics	Bragg's law, structure factor with Waasmaier-Kirfel form factors, multiplicities, Lorentz-polarisation, Debye-Waller, K-alpha doublet, Caglioti resolution, Scherrer broadening, Poisson noise
Repository	https://github.com/OBODA0/quantum-physics

1. The instrument

The simulator models the chain in Figure 1. A copper tube emits the K-alpha doublet at 1.540598 and 1.544426 angstrom, with K-beta removed by a filter or a monochromator. The beam leaves a divergence slit, is collimated by Soller slits, strikes a flat powder specimen on a theta/2theta goniometer, and reaches the detector through a receiving slit. The measured variable is 2theta; the specimen sits at theta, so a reflection appears when 2 d sin(theta) = n lambda.

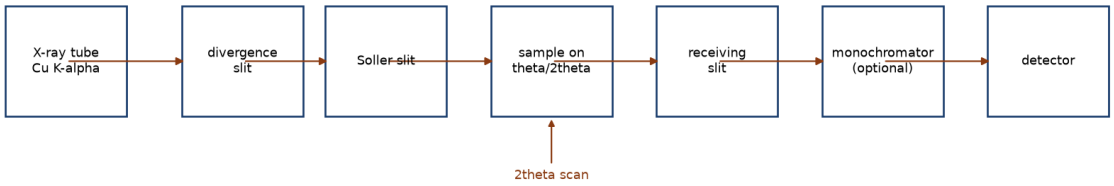


Figure 1: the chain the simulator models. Every box is a module: the tube and lines in `sources.py`*, the slits and flat-plate geometry in* `geometry.py`*, the specimen in* `samples.py`*, the detector in* `detector.py`*, and the scan in* `diffractometer.py`*.*

2. The model

Positions come from Bragg's law and the cell. Intensities are relative, as in assignment 1: the model reproduces patterns and intensity ratios, not absolute radiometry. For each reflection the relative intensity is |F|^2 m LP, where m is the powder multiplicity and LP = (1 + cos^2 2theta) / (sin^2 theta cos theta) without a monochromator. The structure factor sums the basis with the Waasmaier-Kirfel form factor f0(s), s = sin(theta)/lambda, and a Debye-Waller term exp(-B s^2).

Each line is drawn with a pseudo-Voigt profile whose FWHM follows the Caglioti form sqrt(U tan^2 theta + V tan theta + W), combined with the Scherrer width K lambda / (D cos theta) for a crystallite size D. Counts are Poisson draws around the expected mean; the detector's dead time is available as a control.

The model's absences are not tabulated. The 222 line of diamond, the 100 and 110 of an FCC metal and the 100 and 111 of a BCC metal are zero because the summed phases cancel, which is a check that the basis is right.

3. The phase database

Key	Phase	Space group	Cell (A)	Strongest line
si	Silicon (Si)	Fd-3m (227)	a = 5.4311	111 at 28.44 deg
al	Aluminium (Al)	Fm-3m (225)	a = 4.0495	111 at 38.47 deg
cu	Copper (Cu)	Fm-3m (225)	a = 3.6149	111 at 43.32 deg
fe	alpha-Iron (Fe)	Im-3m (229)	a = 2.8665	110 at 44.67 deg
nacl	Rock salt (NaCl)	Fm-3m (225)	a = 5.6402	200 at 31.70 deg
mgo	Periclase (MgO)	Fm-3m (225)	a = 4.2112	200 at 42.92 deg
zno	Zincite (ZnO)	P63mc (186)	a = 3.2494, c = 5.2038	101 at 36.26 deg
tio2-rutile	Rutile (TiO2)	P42/mnm (136)	a = 4.5937, c = 2.9587	110 at 27.44 deg
tio2-anatase	Anatase (TiO2)	I41/amd (141)	a = 3.7845, c = 9.5143	101 at 25.31 deg

Table 1: the phases the simulator ships. The last column is the line the model predicts, which can differ from a printed card by sources and their confidence are listed in `data/SOURCES.md`*; every row is checked against its published strongest reflection in the test suite.*

4. Results

Figure 2 shows the nine phases under the same instrument settings. The differences are the physics: an FCC metal has few strong lines, a rocksalt has a weak 111 because the two form factors nearly cancel, wurtzite breaks the cubic degeneracy, and the two titania polymorphs give different patterns from the same formula.

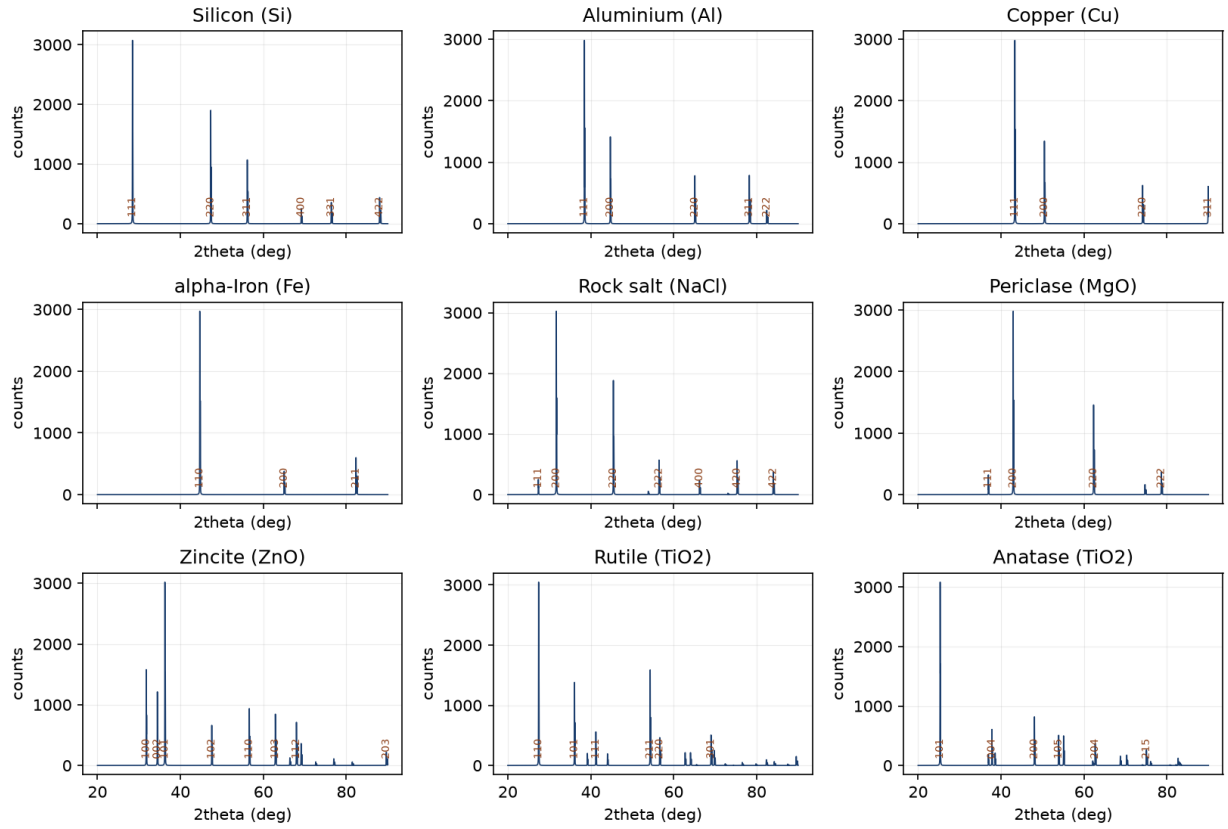


Figure 2: simulated patterns, 20-90 deg, Cu K-alpha, 20 000 counts at the strongest line, no noise. The hkl labels are the reflections the model predicts.

4.1 Crystallite size

Broadening combines the instrument and the specimen in quadrature. Figure 3 fixes the instrument (U, V, W) and varies the crystallite size; the Scherrer analysis in `xrdsim/analysis.py` subtracts the instrumental width in quadrature and recovers the input size to within about 1 % on a noiseless scan (30.2-30.4 nm for an input of 30 nm).

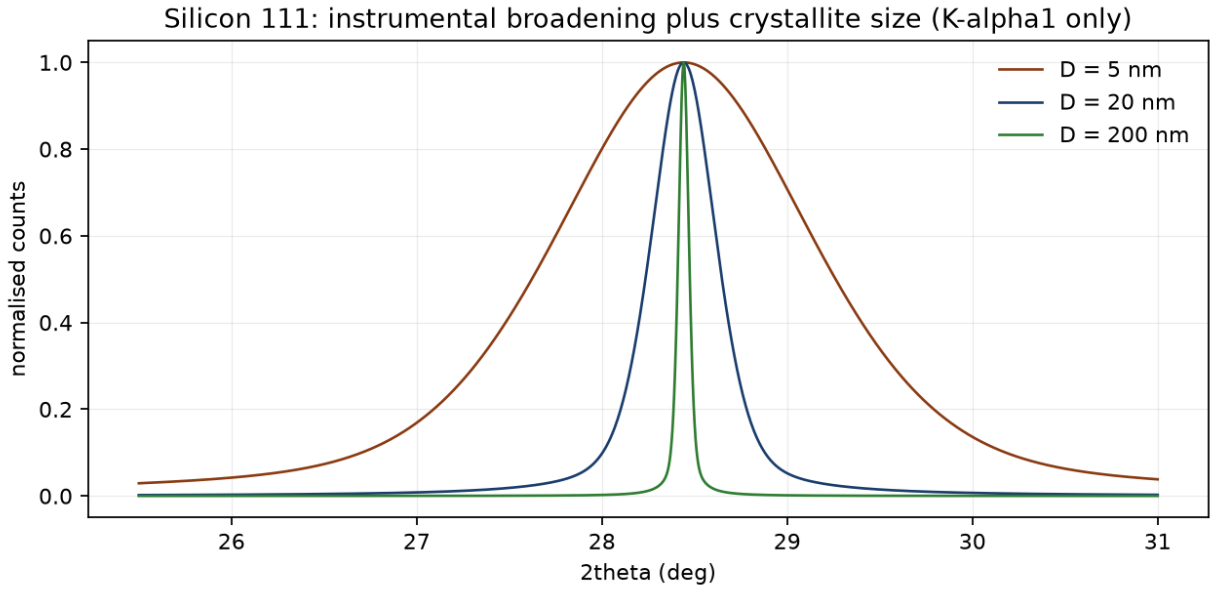


Figure 3: the silicon 111 line at 5, 20 and 200 nm. The curves are normalised; the K-alpha2 line is switched off so that only the intrinsic width remains.

4.2 The K-alpha doublet

The doublet separation grows with angle, Delta(2theta) = 2 tan(theta) (lambda2 - lambda1)/lambda1: about 0.07 deg at 28 deg and 0.28 deg at 88 deg. Whether the two lines resolve depends on the peak width. With the default Caglioti width here (0.05-0.07 deg) silicon's 111 pair is barely separated and every higher line is resolved; a broader instrument merges them into one asymmetric peak, which is why a fitted width at low angle is not the instrument width.

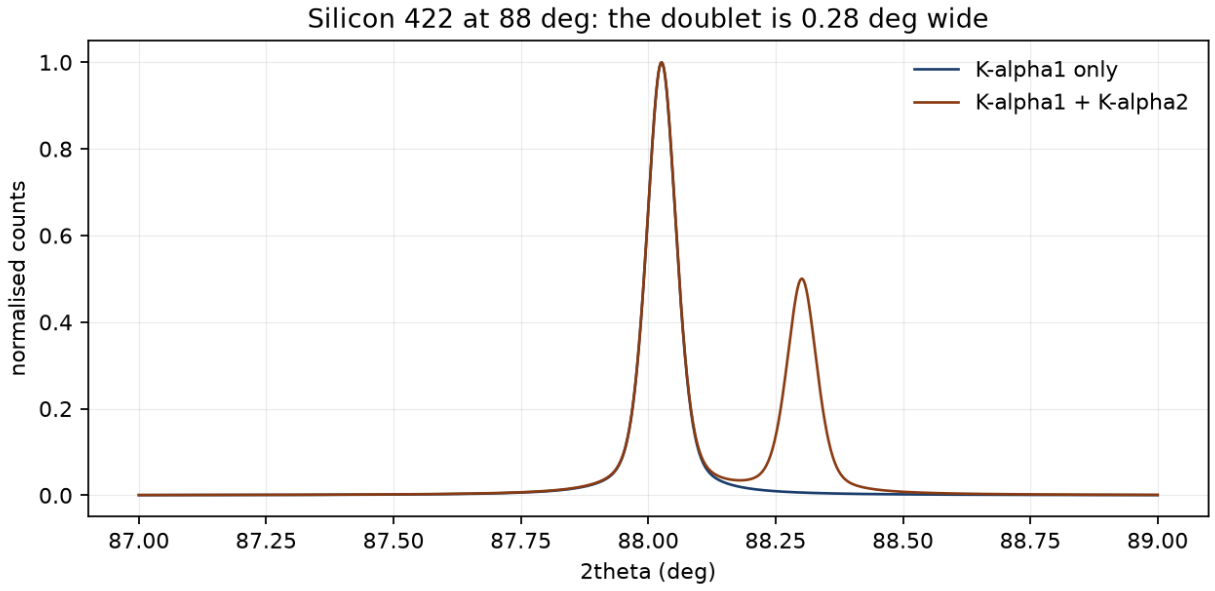


Figure 4: silicon 422 with and without the K-alpha2 line, at the same instrument settings.

5. Verification

The suite covers lattice geometry, profiles, the detector, the structure factor and its absences, the scan, the analysis and the command line interface. Independent checks compare simulated peak positions with certified and published values: silicon 111/220/311/400 at 28.441/47.301/56.120/69.127 deg against the NIST SRM 640d values 28.4425/47.3034/56.1211/69.1295, and the relative intensities 100/64/37/10 against the standard 100/55/30/5. The positions match the certificate to three decimals; the intensity ratios run high at the back of the scan because no absorption correction is applied and every site shares one Debye-Waller factor.

Multiplicities are checked against a brute-force enumeration of the symmetry operations for cubic, tetragonal and hexagonal families. Intensity ratios agree with published patterns to within about a third on strong lines: copper 111/200/220 comes out at 100/46/24 against the published 100/46/20, and rutile 110/101/211 at 100/45/55 against 100/50/60. The deviations are largest on weak lines and are not tuned: every site shares one Debye-Waller factor and no absorption correction is applied. The whole assignment rebuilds from a fresh clone: uv sync, the test suite, figures, tables, PDF and both gates.

Defects found and fixed during the build are listed in `VERIFICATION.md`: a missing step factor in the profile, duplicate reflections from permuted indices, a Lorentz-polarisation divergence at 180 deg, cell constants taken from primitive-cell conversions, and a hand-written anatase basis that produced a forbidden 001 line.

6. What is not modelled

Rietveld refinement, anomalous dispersion, preferred orientation, K-beta residual, detector energy discrimination and absolute intensities. The Cu K-alpha wavelengths are provisional: no primary table could be read from the build host. No simulated pattern has yet been compared with a measured scan; the comparisons above are with published peak positions and intensity ratios.

Appendix A: the code

Every module the simulator ships, in alphabetical order. The same files are in the repository under assignments/02-xrd-diffractometer/xrdsim/.

xrdsim/__init__.py

```
"""XRD diffractometer simulator (assignment 2, quantum physics section)."""
__version__ = "0.1.0"
```

xrdsim/__main__.py

```
from .cli import main

raise SystemExit(main())
```

xrdsim/analysis.py

```
"""Reading a simulated or measured pattern: peak finding, widths and indexing.

The functions here work on arrays (2theta, counts) and make no assumption about where
they came from, so the same code is used on simulator output and on a measured scan.
"""

from __future__ import annotations

import math
from dataclasses import dataclass

import numpy as np

from .profile import scherrer_size
@dataclass(frozen=True)
```

```
class Peak:
    two_theta: float
    height: float
    fwhm: float
    area: float


def find_peaks(two_theta: np.ndarray, counts: np.ndarray,
               min_height_fraction: float = 0.02) -> list[Peak]:
    """Local maxima above a fraction of the tallest point, with width and area."""
    x = np.asarray(two_theta, dtype=float)
    y = np.asarray(counts, dtype=float)
    if x.size != y.size or x.size < 3:
        raise ValueError("need matching arrays of at least three points")
    threshold = min_height_fraction * y.max()
    peaks: list[Peak] = []
    for i in range(1, len(y) - 1):
        if not (y[i] >= y[i - 1] and y[i] > y[i + 1] and y[i] >= threshold):
            continue
        centre, height = _refine(x, y, i)
        half = (height + _local_floor(x, y, i)) / 2.0
        fwhm = _width_at(x, y, i, half)
        area = _area(x, y, i, 4.0 * fwhm if fwhm > 0 else x[1] - x[0])
        peaks.append(Peak(centre, height, fwhm, area))
    return peaks


def _local_floor(x: np.ndarray, y: np.ndarray, i: int, reach_deg: float = 1.0) -> float:
    """Lowest point within `reach_deg` of the peak, so the result does not depend on the step."""
    step = x[1] - x[0] if len(x) > 1 else 1.0
    reach = max(1, int(reach_deg / step))
    left = max(0, i - reach)
    right = min(len(y), i + reach)
    return float(min(y[left:i + 1].min(), y[i:right].min()))


def _width_at(x: np.ndarray, y: np.ndarray, i: int, half: float) -> float:
    left = i
    while left > 0 and y[left] > half:
        left -= 1
    right = i
    while right < len(y) - 1 and y[right] > half:
        right += 1
    if left == i or right == i:
        return 0.0
    x_left = np.interp(half, [y[left], y[left + 1]], [x[left], x[left + 1]])
    x_right = np.interp(half, [y[right], y[right - 1]], [x[right], x[right - 1]])
    return float(x_right - x_left)


def _area(x: np.ndarray, y: np.ndarray, i: int, half_window: float) -> float:
    window = np.abs(x - x[i]) <= half_window
    return float(np.trapezoid(y[window] - np.median(y[window]), x[window]))


def pure_width(observed: float, instrumental: float) -> float:
    """Sample-only FWHM by Gaussian quadrature subtraction, guarded at zero."""
    value = observed * observed - instrumental * instrumental
    return math.sqrt(value) if value > 0 else 0.0


def scherrer_from_peak(peak: Peak, instrumental_fwhm: float, wavelength: float,
                       k: float = 0.9) -> float:
    """Crystallite size in angstrom from one peak, corrected for instrumental width."""
```

```
    beta = pure_width(peak.fwhm, instrumental_fwhm)
    if beta <= 0:
        raise ValueError("the peak is not broader than the instrument")
    return scherrer_size(beta, peak.two_theta, wavelength, k)


def index_cubic(two_thetas: list[float],
                max_n_squared: int = 30, tolerance: float = 0.02) -> list[tuple[tuple[int, int, int], float]]:
    """Assign (hkl) to a cubic pattern by the sin^2(theta) ratios.

    The cell returned is the smallest one that fits, so a body-centred pattern comes back
    as a primitive cell with half the conventional edge; `lattice_hint` names that.

    Returns the assignments and the implied n^2 = h^2 + k^2 + l^2. The first reflection
    is taken as the lowest-angle one; every other angle must fall on an integer multiple
    of its sin^2(theta) to within `tolerance` (relative).
    """
    if not two_thetas:
        raise ValueError("no peaks to index")
    two_thetas = sorted(two_thetas)
    s2 = [math.sin(math.radians(t / 2.0)) ** 2 for t in two_thetas]
    base = s2[0]
    for n2_first in range(1, max_n_squared + 1):
        ratios = [value / base * n2_first for value in s2]
        if all(abs(r - round(r)) <= tolerance * max(round(r), 1) for r in ratios):
            assignments = []
            for angle, ratio in zip(two_thetas, ratios):
                n2 = int(round(ratio))
                assignments.append((_hkl_from_n2(n2), angle))
            return assignments
    raise ValueError("the pattern does not index as cubic")


def lattice_hint(n_squared: list[int]) -> str:
    """Describe the lattice from the observed n^2 sequence.

    All-even n^2 means a body-centred lattice described on half the cell; the FCC sequence
    starts 3, 4, 8, 11, 12, 16, 19, 20, 24. Anything else is primitive.
    """
    values = sorted({int(value) for value in n_squared})
    fcc = {3, 4, 8, 11, 12, 16, 19, 20, 24, 27, 32, 35, 36, 40, 43, 44, 48}
    if values and all(value % 2 == 0 for value in values):
        return "body-centred, conventional cell twice the refined edge"
    if set(values) <= fcc:
        return "face-centred (the 3, 4, 8, 11 sequence)"
    if 1 in values:
        return "primitive cell; a body-centred sample would have twice this edge"
    return "primitive lattice"


def _hkl_from_n2(n2: int) -> tuple[int, int, int]:
    for h in range(0, 8):
        for k in range(0, 8):
            l_sq = n2 - h * h - k * k
            if l_sq < 0:
                continue
            l = round(math.isqrt(l_sq))
            if l * l == l_sq:
                return tuple(sorted((h, k, l), reverse=True))
    raise ValueError(f"no (hkl) has n^2 = {n2}")


def _refine(x: np.ndarray, y: np.ndarray, i: int) -> tuple[float, float]:
    """Parabolic refinement of a sampled maximum: position and height between bins."""
```

```
    y0, y1, y2 = y[i - 1], y[i], y[i + 1]
    denom = y0 - 2.0 * y1 + y2
    if denom == 0:
        return float(x[i]), float(y[i])
    delta = 0.5 * (y0 - y2) / denom
    if abs(delta) > 0.5:
        return float(x[i]), float(y[i])
    return float(x[i] + delta * (x[i + 1] - x[i])), float(y1 - 0.25 * (y0 - y2) * delta)


def williamson_hall(peaks: list[Peak], instrumental_fwhm_for,
                    wavelength: float, k: float = 0.9) -> tuple[float, float]:
    """Crystallite size and microstrain from the Williamson-Hall plot.

    beta cos(theta) = K lambda / D + 4 epsilon sin(theta), fitted by least squares over the
    peaks given. `instrumental_fwhm_for(two_theta)` returns the instrument width at that
    angle. Returns (size in angstrom, strain) and raises when fewer than two peaks are
    usable.
    """
    xs, ys = [], []
    for peak in peaks:
        beta = pure_width(peak.fwhm, instrumental_fwhm_for(peak.two_theta))
        if beta <= 0 or peak.two_theta <= 0:
            continue
        theta = math.radians(peak.two_theta / 2.0)
        xs.append(4.0 * math.sin(theta))
        ys.append(math.radians(beta) * math.cos(theta))
    if len(xs) < 2:
        raise ValueError("Williamson-Hall needs at least two peaks broader than the instrument")
    slope, intercept = np.polyfit(xs, ys, 1)
    if intercept <= 0:
        raise ValueError("the fit has no positive intercept, so no size can be reported")
    return k * wavelength / intercept, float(slope)


def identify_phase(two_thetas: list[float], phase, instrument,
                   tolerance: float = 0.15,
                   min_intensity_fraction: float = 0.15) -> float:
    """Fraction of a phase's strong reflections that the observed peaks contain.

    A score of 1.0 means every predicted line above `min_intensity_fraction` of the
    phase's strongest has an observed peak within `tolerance` degrees.
    """
    from .diffractometer import reflections
    rows = reflections(phase, instrument)
    if not rows:
        return 0.0
    strongest = max(row.intensity for row in rows)
    predicted = [row.two_theta for row in rows
                 if row.intensity > min_intensity_fraction * strongest]
    if not predicted or not two_thetas:
        return 0.0
    matched = sum(1 for angle in predicted
                  if min(abs(angle - seen) for seen in two_thetas) <= tolerance)
    return matched / len(predicted)


def identify(two_thetas: list[float], phases: dict, instrument,
             tolerance: float = 0.15) -> list[tuple[str, float]]:
    """Rank the phases of a database by how well they explain the observed peak positions."""
    scored = [(key, identify_phase(two_thetas, phase, instrument, tolerance))
              for key, phase in phases.items()]
    return sorted(scored, key=lambda row: (-row[1], row[0]))
```

xrdsim/atoms.py

```
  """Atomic scattering factors and the Debye-Waller term.

  `f0` is interpolated from data/scattering_factors.json, sampled once from the
  Waasmaier-Kirfel analytic form factors (Acta Cryst. A51, 416-431, 1995).
  `s = sin(theta) / lambda` in 1/angstrom.
  """

  from __future__ import annotations

  import json
  import math
  import pathlib

  DATA_PATH = pathlib.Path(__file__).resolve().parent.parent / "data" / "scattering_factors.json"

  _table: dict | None = None


  def _load() -> dict:
      global _table
      if _table is None:
          payload = json.loads(DATA_PATH.read_text(encoding="utf-8"))
          _table = {
              "s_max": payload["grid_s_inv_A"][-1],
              "step": payload["grid_s_inv_A"][1] - payload["grid_s_inv_A"][0],
              "elements": payload["f0"],
          }
      return _table


  def f0(element: str, s: float) -> float:
      """Elastic form factor f0 in electrons. `s = sin(theta)/lambda` in 1/angstrom."""
      table = _load()
      values = table["elements"].get(element)
      if values is None:
          raise KeyError(f"no scattering factor for {element!r}; rebuild data/scattering_factors.json")
      if s < 0:
          raise ValueError("s must be non-negative")
      position = s / table["step"]
      last = len(values) - 1
      if position >= last:
          return float(values[last])
      i = int(position)
      frac = position - i
      return float(values[i] * (1.0 - frac) + values[i + 1] * frac)


  def debye_waller(b_iso: float, s: float) -> float:
      """Temperature factor exp(-B s^2), with B = 8 pi^2 <u^2> in square angstrom."""
      if b_iso < 0:
          raise ValueError("B must be non-negative")
      return math.exp(-b_iso * s * s)
```

xrdsim/cli.py

```
  """Command line interface: `python -m xrdsim`."""

  from __future__ import annotations

  import argparse
  import pathlib
  from . import sources
```

```
from .analysis import (find_peaks, identify, index_cubic, lattice_hint,
                       scherrer_from_peak, williamson_hall)
from .diffractometer import Instrument, reflections, scan
from .profile import caglioti_fwhm
from .samples import PHASES, get_phase


def _instrument(args) -> Instrument:
    return Instrument(
        ka2=not args.no_ka2,
        monochromator_two_theta=args.monochromator,
        reference_counts=args.counts,
        background_rate=args.background,
        dwell_s=args.dwell,
        rng_seed=None if args.noise == 0 else args.noise,
        sample_displacement_mm=args.displacement,
    )


def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="xrdsim", description=__doc__)
    parser.add_argument("--phase", help="phase key (see --list-phases)")
    parser.add_argument("--range", nargs=2, type=float, metavar=("MIN", "MAX"),
                        default=[20.0, 90.0], help="2theta scan range in degrees")
    parser.add_argument("--step", type=float, default=0.02, help="2theta step in degrees")
    parser.add_argument("--dwell", type=float, default=1.0, help="counting time per step, s")
    parser.add_argument("--counts", type=float, default=20_000.0,
                        help="integrated counts at the strongest reflection")
    parser.add_argument("--background", type=float, default=0.5, help="background, counts/s")
    parser.add_argument("--displacement", type=float, default=0.0, metavar="MM",
                        help="sample surface displaced from the goniometer axis")
    parser.add_argument("--size", type=float, default=None,
                        help="crystallite size in angstrom (adds size broadening)")
    parser.add_argument("--no-ka2", action="store_true", help="drop the K-alpha2 line")
    parser.add_argument("--monochromator", type=float, default=None,
                        help="monochromator 2theta in degrees")
    parser.add_argument("--noise", type=int, default=0,
                        help="Poisson seed; 0 disables counting noise")
    parser.add_argument("--out", type=pathlib.Path, help="write the scan to this image")
    parser.add_argument("--table", action="store_true", help="print the reflection table")
    parser.add_argument("--index", action="store_true",
                        help="index the observed peaks as a cubic pattern")
    parser.add_argument("--scherrer", action="store_true",
                        help="report Scherrer sizes for the observed peaks")
    parser.add_argument("--identify", action="store_true",
                        help="rank the phase database against the simulated pattern")
    parser.add_argument("--list-phases", action="store_true", help="list the phase database")
    args = parser.parse_args(argv)

    if args.list_phases:
        for key, phase in sorted(PHASES.items()):
            print(f"{key:14s} {phase.formula:8s} {phase.space_group:18s} {phase.source}")
        return 0
    if not args.phase:
        parser.error("--phase is required unless --list-phases is given")

    try:
        return _run(args)
    except (KeyError, ValueError) as error:
        parser.error(str(error).strip('"'))


def _run(args) -> int:
        phase = get_phase(args.phase)
```

```
instrument = _instrument(args)
result = scan(phase, args.range[0], args.range[1], args.step, instrument,
              size_a=args.size)

if args.table:
    strongest = max(row.intensity for row in result.reflections)
    print(f"{'hkl':>10s} {'d (A)':>8s} {'2theta':>8s} {'m':>3s} {'I/I0':>7s}")
    for row in result.reflections:
        if row.intensity < 1e-4 * strongest:
            continue
        label = "".join(str(v) for v in row.hkl)
        print(f"{label:>10s} {row.d:8.4f} {row.two_theta:8.3f} {row.multiplicity:3d} "
              f"{100 * row.intensity / strongest:7.2f}")
    return 0

peaks = find_peaks(result.two_theta, result.counts)
print(f"phase      : {phase.name} ({phase.formula}), {phase.space_group}")
print(f"wavelength : {instrument.wavelength:.6f} A"
      + (" + K-alpha2" if instrument.ka2 else ""))
print(f"range      : {args.range[0]} - {args.range[1]} deg, step {args.step}")
print(f"peaks      : {len(peaks)}")
for peak in peaks[:12]:
    line = f"  {peak.two_theta:8.3f} deg  height {peak.height:9.0f}  FWHM {peak.fwhm:.4f}"
    if args.size and not args.scherrer:
        try:
            size = scherrer_from_peak(peak, caglioti_fwhm(peak.two_theta, *instrument.caglioti),
                                      instrument.wavelength)
            line += f"  Scherrer {size / 10.0:.1f} nm"
        except ValueError:
            pass
    print(line)

if args.index:
    try:
        rows = index_cubic([peak.two_theta for peak in peaks])
    except ValueError as error:
        print(f"indexing failed: {error}")
    else:
        print("indexing   : hkl            2theta")
        for hkl, angle in rows:
            print(f"             {str(hkl):14s} {angle:7.3f}")
        n_squared = [hkl[0] ** 2 + hkl[1] ** 2 + hkl[2] ** 2 for hkl, _ in rows]
        print(f"lattice    : {lattice_hint(n_squared)}")

if args.scherrer:
    print("Scherrer   : peak            size (nm)")
    for peak in peaks:
        try:
            size = scherrer_from_peak(
                peak, caglioti_fwhm(peak.two_theta, *instrument.caglioti),
                instrument.wavelength)
        except ValueError:
            continue
        print(f"             {peak.two_theta:7.3f}         {size / 10.0:6.1f}")
    try:
        wh_size, wh_strain = williamson_hall(
            peaks, lambda angle: caglioti_fwhm(angle, *instrument.caglioti),
            instrument.wavelength)
    except ValueError as error:
        print(f"Williamson-Hall: {error}")
    else:
        print(f"Williamson-Hall: D = {wh_size / 10.0:.1f} nm, strain = {wh_strain:.5f}"
              + ("" if args.no_ka2 else " (use --no-ka2: the shoulders bias the fit)"))
if args.identify:
```

```
            ranking = identify([peak.two_theta for peak in peaks], PHASES, instrument)
            print("identification: phase          score")
            for key, score in ranking[:5]:
                print(f"               {key:14s} {score:5.2f}")

        if args.out:
            from .plot import save_scan
            args.out.parent.mkdir(parents=True, exist_ok=True)
            save_scan(result, args.out,
                      title=f"{phase.name}, {instrument.wavelength:.4f} A")
            print(f"wrote {args.out}")
        return 0
```

xrdsim/detector.py

```
"""Detector: counting statistics and dead time.

A powder scan is a counting experiment. The model returns expected counts per step; the
detector turns them into observed counts with Poisson noise.
"""

from __future__ import annotations

import math


def expected_counts(intensity: float, dwell_s: float) -> float:
    """Mean counts at a step: intensity (counts per second) times dwell time."""
    if dwell_s <= 0:
        raise ValueError("dwell time must be positive")
    return max(intensity, 0.0) * dwell_s


def observe(mean_counts: float, rng) -> float:
    """One Poisson draw for a step. `rng` is a numpy Generator."""
    return float(rng.poisson(max(mean_counts, 0.0)))


def count_noise(mean_counts: float) -> float:
    """Standard deviation of a Poisson count (sqrt(N)); zero for an empty step."""
    return math.sqrt(max(mean_counts, 0.0))


def apply_dead_time(true_rate: float, tau_s: float) -> float:
    """Observed rate for a non-paralysable detector: n = m / (1 + m tau)."""
    if true_rate < 0 or tau_s < 0:
        raise ValueError("rates and dead time must be non-negative")
    return true_rate / (1.0 + true_rate * tau_s)
```

xrdsim/diffractometer.py

```
"""The diffractometer: reflections, peak profiles and the scan itself.

A scan is built in counts per step. Reflection intensities are relative (I/I0), as in
Assignment 1: the model reproduces patterns and intensity ratios, not absolute
radiometry.
"""

from __future__ import annotations

import math
from dataclasses import dataclass, field

import numpy as np
from . import sources
```

```
from .detector import apply_dead_time, expected_counts
from .lattice import d_spacing
from .profile import (add_in_quadrature, caglioti_fwhm, pseudo_voigt_array,
                      size_fwhm)
from .samples import Phase
from .structure import lorentz_polarisation, multiplicity, structure_factor


@dataclass(frozen=True)
class Instrument:
    wavelength: float = sources.CU_KA1
    ka2: bool = True
    monochromator_two_theta: float | None = None
    caglioti: tuple[float, float, float] = (0.004, -0.002, 0.003)
    eta: float = 0.5
    reference_counts: float = 20_000.0     # counts at the strongest reflection
    background_rate: float = 0.5           # counts per second, flat
    dwell_s: float = 1.0
    dead_time_s: float = 0.0
    rng_seed: int | None = None      # None: no counting noise; an int seeds the Poisson draws
    max_two_theta: float = 150.0      # no goniometer reaches 180 deg, and LP diverges there
    sample_displacement_mm: float = 0.0  # specimen surface off the goniometer axis
    goniometer_radius_mm: float = 240.0


@dataclass(frozen=True)
class Reflection:
    hkl: tuple[int, int, int]
    d: float
    two_theta: float
    multiplicity: int
    f_squared: float
    intensity: float


@dataclass
class Scan:
    two_theta: np.ndarray
    counts: np.ndarray
    reflections: list[Reflection] = field(default_factory=list)
    phase: Phase | None = None
    instrument: Instrument | None = None



def reflections(phase: Phase, instrument: Instrument, hmax: int = 8) -> list[Reflection]:
    """Every reflection the wavelength reaches, with its relative intensity."""
    rows: list[Reflection] = []
    for h in range(0, hmax + 1):
        for k in range(0, hmax + 1):
            for l in range(0, hmax + 1):
                if h == 0 and k == 0 and l == 0:
                    continue
                if not _representative(h, k, l, phase.system):
                    continue
                try:
                    d = d_spacing(h, k, l, **phase.cell_kwargs())
                except ValueError:
                    continue
                if 2.0 * d < instrument.wavelength:
                    continue
                two_theta = 2.0 * math.degrees(
                    math.asin(instrument.wavelength / (2.0 * d)))
                if two_theta > instrument.max_two_theta:
```

```
                    continue
                f = structure_factor(phase, h, k, l, instrument.wavelength)
                m = multiplicity(h, k, l, phase.system)
                lp = lorentz_polarisation(two_theta, instrument.monochromator_two_theta)
                rows.append(Reflection((h, k, l), d, two_theta, m, abs(f) ** 2,
                                       abs(f) ** 2 * m * lp))
    rows.sort(key=lambda r: r.two_theta)
    return rows


def scan(phase: Phase, two_theta_min: float, two_theta_max: float, step: float,
         instrument: Instrument | None = None, size_a: float | None = None,
         hmax: int = 8) -> Scan:
    """Simulate one 2theta scan, with Poisson noise and optional crystallite size."""
    if instrument is None:
        instrument = Instrument()
    if step <= 0 or two_theta_max <= two_theta_min:
        raise ValueError("the scan range and step must be positive")
    grid = np.arange(two_theta_min, two_theta_max + step / 2.0, step)
    rows = reflections(phase, instrument, hmax)
    if not rows:
        raise ValueError("no reachable reflections in this range")

    strongest = max(row.intensity for row in rows)
    if strongest <= 0:
        raise ValueError("every reflection has zero intensity")

    lines = [(instrument.wavelength, 1.0)]
    if instrument.ka2:
        # the doublet is scaled from the instrument's own K-alpha1 so that another anode
        # keeps its own wavelengths instead of silently getting copper's
        ka2 = instrument.wavelength * sources.CU_KA2 / sources.CU_KA1
        lines = [(instrument.wavelength, 1.0), (ka2, sources.KA2_TO_KA1)]

    mean = np.full_like(grid, expected_counts(instrument.background_rate, instrument.dwell_s),
                        dtype=float)
    for row in rows:
        area_total = instrument.reference_counts * row.intensity / strongest
        for wavelength, weight in lines:
            d = d_spacing(*row.hkl, **phase.cell_kwargs())
            centre = 2.0 * math.degrees(math.asin(min(1.0, wavelength / (2.0 * d))))
            centre += displacement_shift(centre, instrument.sample_displacement_mm,
                                         instrument.goniometer_radius_mm)
            width = caglioti_fwhm(centre, *instrument.caglioti)
            if size_a is not None:
                width = add_in_quadrature(width, size_fwhm(size_a, centre, wavelength))
            if not (two_theta_min - 5.0 * width <= centre <= two_theta_max + 5.0 * width):
                continue
            profile = pseudo_voigt_array(grid, centre, width, instrument.eta)
            mean += area_total * weight * profile * step

    if instrument.dead_time_s:
        rate = mean / instrument.dwell_s
        mean = np.array([apply_dead_time(r, instrument.dead_time_s) * instrument.dwell_s
                         for r in rate])

    if instrument.rng_seed is None:
        counts = mean
    else:
        rng = np.random.default_rng(instrument.rng_seed)
        counts = rng.poisson(np.clip(mean, 0.0, None)).astype(float)

    return Scan(grid, counts, rows, phase, instrument)
def _representative(h: int, k: int, l: int, system: str) -> bool:
```

```
    """True for one member of each symmetry family.

    Enumerating h, k, l >= 0 visits every permutation of a reflection; the multiplicity
    already counts the whole orbit, so only the canonical member may be used.
    """
    if system == "cubic":
        return h >= k >= l
    if system == "tetragonal":
        return h >= k
    if system == "hexagonal":
        return h >= k
    return True


def displacement_shift(two_theta_deg: float, displacement_mm: float,
                       radius_mm: float) -> float:
    """2theta shift in degrees for a flat specimen displaced from the goniometer axis.

    Delta(2theta) = -2 s cos(theta) / R, in radians.
    """
    if displacement_mm == 0.0:
        return 0.0
    if radius_mm <= 0:
        raise ValueError('the goniometer radius must be positive')
    theta = math.radians(two_theta_deg / 2.0)
    return math.degrees(-2.0 * displacement_mm * math.cos(theta) / radius_mm)
```

xrdsim/geometry.py

```
"""Bragg-Brentano geometry: sample illumination and absorption.

Angles in degrees, lengths in mm, mass attenuation in cm^2/g. The flat-plate,
infinite-thickness case is the one a laboratory powder scan approximates.
"""

from __future__ import annotations

import math


def bragg_angle(two_theta_deg: float) -> float:
    """theta for a given 2theta."""
    return two_theta_deg / 2.0


def footprint_length(two_theta_deg: float, divergence_slit_mm: float,
                     sample_length_mm: float = 20.0) -> float:
    """Length of sample illuminated by the beam, in mm.

    The beam leaves the divergence slit with a fixed width, so the footprint on the
    flat plate grows as 1/sin(theta) and is clipped by the sample holder.
    """
    theta = math.radians(bragg_angle(two_theta_deg))
    if theta <= 0:
        raise ValueError("2theta must be positive")
    return min(sample_length_mm, divergence_slit_mm / math.sin(theta))


def linear_attenuation(density_g_cm3: float, mass_attenuation_cm2_g: float) -> float:
    """mu in 1/cm."""
    if density_g_cm3 <= 0 or mass_attenuation_cm2_g <= 0:
        raise ValueError("density and mass attenuation must be positive")
    return density_g_cm3 * mass_attenuation_cm2_g
def is_infinitely_thick(mu_per_cm: float, thickness_cm: float, threshold: float = 0.01) -> bool:
```

```
    """True when exp(-2 mu t / sin(theta)) is negligible; here the worst case theta -> 90 deg."""
    return math.exp(-2.0 * mu_per_cm * thickness_cm) < threshold


def flat_plate_absorption(mu_per_cm: float) -> float:
    """Absorption factor for a thick flat plate: A = 1 / (2 mu), independent of theta."""
    if mu_per_cm <= 0:
        raise ValueError("mu must be positive")
    return 1.0 / (2.0 * mu_per_cm)


def penetration_depth(mu_per_cm: float, two_theta_deg: float, fraction: float = 0.99) -> float:
    """Depth in cm that carries `fraction` of the diffracted intensity."""
    if not 0.0 < fraction < 1.0:
        raise ValueError("fraction must lie in (0, 1)")
    theta = math.radians(bragg_angle(two_theta_deg))
    return -math.log(1.0 - fraction) * math.sin(theta) / (2.0 * mu_per_cm)
```

xrdsim/lattice.py

```
"""Lattice geometry: d-spacings, Bragg angles and lattice-parameter refinement.

Lengths are in angstrom, angles in degrees. Only the settings a powder scan needs are
supported here: cubic, tetragonal, hexagonal and orthorhombic.
"""

from __future__ import annotations

import math


def d_spacing(h: int, k: int, l: int, *, a: float, b: float | None = None,
              c: float | None = None, hexagonal: bool = False) -> float:
    """Interplanar spacing in angstrom.

    `hexagonal=True` selects the hexagonal setting (a = b, gamma = 120 degrees).
    Otherwise `b` and `c` both None is cubic, `c` given with `b` None is tetragonal, and
    both `b` and `c` given is orthorhombic. A lone `b` is rejected rather than guessed.
    """
    if a <= 0:
        raise ValueError("lattice parameter must be positive")
    if hexagonal:
        if c is None:
            raise ValueError("the hexagonal setting needs c")
        inv_sq = 4.0 / 3.0 * (h * h + h * k + k * k) / a**2 + l * l / c**2
    elif b is None and c is None:
        inv_sq = (h * h + k * k + l * l) / a**2
    elif b is None:
        inv_sq = (h * h + k * k) / a**2 + l * l / c**2
    elif c is None:
        raise ValueError("an orthorhombic cell needs both b and c; for a tetragonal cell pass c and leave b No
    ne")
    else:
        inv_sq = h * h / a**2 + k * k / b**2 + l * l / c**2
    if inv_sq == 0:
        raise ValueError("(0,0,0) has no d-spacing")
    return 1.0 / math.sqrt(inv_sq)


def two_theta(d: float, wavelength: float) -> float:
    """Bragg angle in degrees for a d-spacing, or ValueError if the reflection is unreachable."""
    if d <= 0:
        raise ValueError("d-spacing must be positive")
    s = wavelength / (2.0 * d)
```

```
    if s > 1.0:
        raise ValueError(
            f"d = {d:.4f} A is unreachable at lambda = {wavelength:.4f} A (needs sin(theta) > 1)")
    return math.degrees(2.0 * math.asin(s))


def d_from_two_theta(two_theta_deg: float, wavelength: float) -> float:
    """d-spacing in angstrom from a measured 2theta, by Bragg's law."""
    theta = math.radians(two_theta_deg / 2.0)
    s = math.sin(theta)
    if s == 0:
        raise ValueError("2theta = 0 carries no information")
    return wavelength / (2.0 * s)


def refine_cubic_a(peaks: list[tuple[float, int, int, int]], wavelength: float) -> float:
    """Least-squares cubic lattice parameter from indexed peaks.

    `peaks` is a list of (two_theta_deg, h, k, l). The fit is an unweighted least squares
    through the origin, sin^2(theta) = (lambda^2 / 4a^2) * (h^2 + k^2 + l^2): no per-peak
    uncertainties, and no zero-offset term, so a displaced specimen biases the result.
    """
    xs, ys = [], []
    for two_theta_deg, h, k, l in peaks:
        n_sq = h * h + k * k + l * l
        if n_sq == 0:
            raise ValueError("(0,0,0) cannot be indexed")
        xs.append(n_sq)
        ys.append(math.sin(math.radians(two_theta_deg / 2.0)) ** 2)
    denom = sum(x * x for x in xs)
    if denom == 0:
        raise ValueError("no peaks given")
    slope = sum(x * y for x, y in zip(xs, ys)) / denom
    if slope <= 0:
        raise ValueError("degenerate peak list")
    return wavelength / (2.0 * math.sqrt(slope))


def peak_positions(hkls, wavelength: float, **cell) -> list[tuple[tuple[int, int, int], float, float]]:
    """(hkl, d, 2theta) for every reflection the wavelength can reach, sorted by 2theta."""
    out = []
    for h, k, l in hkls:
        d = d_spacing(h, k, l, **cell)
        try:
            out.append(((h, k, l), d, two_theta(d, wavelength)))
        except ValueError:
            continue
    return sorted(out, key=lambda row: row[2])
```

xrdsim/plot.py

```
"""Matplotlib rendering of a scan.

Assignment 1 already fixed the house style: a scan is drawn as counts against 2theta,
with the reflections the model predicts marked, and nothing decorated that the data do
not carry.
"""

from __future__ import annotations

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
```

```
from .diffractometer import Scan


def plot_scan(scan: Scan, ax=None, title: str | None = None, annotate: bool = True):
    """Draw one scan; returns the axes."""
    if ax is None:
        _, ax = plt.subplots(figsize=(9, 4.2))
    ax.plot(scan.two_theta, scan.counts, lw=0.8, color="#1a3d6d")
    if annotate and scan.reflections:
        strongest = max(row.intensity for row in scan.reflections)
        placed: list[float] = []
        for row in sorted(scan.reflections, key=lambda r: -r.intensity):
            if row.intensity < 0.08 * strongest:
                continue
            if any(abs(row.two_theta - angle) < 1.2 for angle in placed):
                continue
            placed.append(row.two_theta)
            ax.annotate("".join(str(v) for v in row.hkl), (row.two_theta, 0),
                        textcoords="offset points", xytext=(0, 4),
                        fontsize=6.5, rotation=90, ha="center", va="bottom",
                        color="#8a3b12")
    ax.set_xlabel("2theta (deg)")
    ax.set_ylabel("counts")
    if title:
        ax.set_title(title)
    ax.grid(alpha=0.25, lw=0.5)
    return ax


def save_scan(scan: Scan, path, title: str | None = None, dpi: int = 160) -> None:
    """Write one scan to an image file."""
    fig, ax = plt.subplots(figsize=(9, 4.2))
    plot_scan(scan, ax=ax, title=title or (scan.phase.name if scan.phase else None))
    fig.tight_layout()
    fig.savefig(path, dpi=dpi)
    plt.close(fig)
```

xrdsim/profile.py

```
"""Peak profiles and the instrument resolution function.

The profile is normalised to unit area, so a peak's height follows from its integrated
intensity and its width. Angles are in degrees, widths are FWHM in degrees.
"""

from __future__ import annotations

import math

TWO_LN2 = 2.0 * math.log(2.0)


def gaussian(x: float, center: float, fwhm: float) -> float:
    """Unit-area Gaussian."""
    sigma = fwhm / (2.0 * math.sqrt(TWO_LN2))
    z = (x - center) / sigma
    return math.exp(-0.5 * z * z) / (sigma * math.sqrt(2.0 * math.pi))


def lorentzian(x: float, center: float, fwhm: float) -> float:
    """Unit-area Lorentzian."""
    half = fwhm / 2.0
    return (half / math.pi) / ((x - center) ** 2 + half * half)
def pseudo_voigt(x: float, center: float, fwhm: float, eta: float) -> float:
```

```
    """Unit-area pseudo-Voigt: eta of Lorentzian mixed with (1 - eta) of Gaussian."""
    if not 0.0 <= eta <= 1.0:
        raise ValueError("eta must lie in [0, 1]")
    return eta * lorentzian(x, center, fwhm) + (1.0 - eta) * gaussian(x, center, fwhm)


def caglioti_fwhm(two_theta_deg: float, U: float, V: float, W: float) -> float:
    """Instrumental FWHM in degrees: U tan^2(theta) + V tan(theta) + W."""
    t = math.tan(math.radians(two_theta_deg / 2.0))
    value = U * t * t + V * t + W
    return math.sqrt(max(value, 0.0))


def scherrer_size(fwhm_deg: float, two_theta_deg: float, wavelength: float,
                  k: float = 0.9) -> float:
    """Crystallite size in angstrom from a sample-broadened FWHM (Scherrer).

    `fwhm_deg` must already be corrected for instrumental broadening.
    """
    if fwhm_deg <= 0:
        raise ValueError("FWHM must be positive")
    beta = math.radians(fwhm_deg)
    theta = math.radians(two_theta_deg / 2.0)
    return k * wavelength / (beta * math.cos(theta))


def size_fwhm(size_a: float, two_theta_deg: float, wavelength: float,
              k: float = 0.9) -> float:
    """FWHM in degrees produced by a crystallite of `size_a` angstrom (inverse Scherrer)."""
    if size_a <= 0:
        raise ValueError("size must be positive")
    theta = math.radians(two_theta_deg / 2.0)
    return math.degrees(k * wavelength / (size_a * math.cos(theta)))


def add_in_quadrature(*fwhms: float) -> float:
    """Combine Gaussian widths."""
    return math.sqrt(sum(f * f for f in fwhms))


def pseudo_voigt_array(x, center: float, fwhm: float, eta: float):
    """Unit-area pseudo-Voigt over an array; same numbers as pseudo_voigt, vectorised."""
    import numpy as np
    if not 0.0 <= eta <= 1.0:
        raise ValueError("eta must lie in [0, 1]")
    values = np.asarray(x, dtype=float)
    sigma = fwhm / (2.0 * math.sqrt(TWO_LN2))
    half = fwhm / 2.0
    z = (values - center) / sigma
    gaussian_part = np.exp(-0.5 * z * z) / (sigma * math.sqrt(2.0 * math.pi))
    lorentzian_part = (half / math.pi) / ((values - center) ** 2 + half * half)
    return eta * lorentzian_part + (1.0 - eta) * gaussian_part
```

xrdsim/samples.py

```
"""The phase database: cell, symmetry-expanded basis and the source of each number.

Coordinates are fractions of the cell edge. Every site list below is the full basis of
the conventional cell, already expanded by the space-group operations, so the structure
factor needs no symmetry engine. Lattice constants carry the record they came from.
"""

from __future__ import annotations
from dataclasses import dataclass, field
```

```
@dataclass(frozen=True)
class Site:
    element: str
    x: float
    y: float
    z: float
    occupancy: float = 1.0
    b_iso: float = 0.5          # square angstrom; a stated assumption for every phase


@dataclass(frozen=True)
class Phase:
    key: str
    name: str
    formula: str
    space_group: str
    a: float
    c: float | None = None
    b: float | None = None
    hexagonal: bool = False
    sites: tuple[Site, ...] = field(default_factory=tuple)
    source: str = ""

    @property
    def system(self) -> str:
        if self.hexagonal:
            return "hexagonal"
        if self.b is not None:
            return "orthorhombic"
        return "cubic" if self.c is None else "tetragonal"

    def cell_kwargs(self) -> dict:
        return {"a": self.a, "b": self.b, "c": self.c, "hexagonal": self.hexagonal}

    @property
    def volume(self) -> float:
        """Conventional-cell volume in cubic angstrom."""
        if self.hexagonal:
            return self.a * self.a * self.c * 0.8660254037844386
        if self.b is None and self.c is None:
            return self.a ** 3
        return self.a * (self.b or self.a) * self.c


def _fcc(element: str, b_iso: float = 0.5) -> tuple[Site, ...]:
    return tuple(Site(element, x, y, z, b_iso=b_iso) for x, y, z in
                 [(0, 0, 0), (0, .5, .5), (.5, 0, .5), (.5, .5, 0)])


def _rocksalt(cation: str, anion: str) -> tuple[Site, ...]:
    sites = list(_fcc(cation))
    sites += [Site(anion, x + .5, y, z) for x, y, z in
              [(0, 0, 0), (0, .5, .5), (.5, 0, .5), (.5, .5, 0)]]
    return tuple(sites)


PHASES: dict[str, Phase] = {
    "si": Phase(
        key="si", name="Silicon", formula="Si", space_group="Fd-3m (227)",
        a=5.431144, sites=tuple(
            [Site("Si", x, y, z) for x, y, z in
             [(0, 0, 0), (0, .5, .5), (.5, 0, .5), (.5, .5, 0),
              (.25, .25, .25), (.25, .75, .75), (.75, .25, .75), (.75, .75, .25)]]),
        source="NIST SRM 640f certificate, a = 5.431144(8) A at 22.5 C"),
```

```
    "al": Phase(
        key="al", name="Aluminium", formula="Al", space_group="Fm-3m (225)",
        a=4.0495, sites=_fcc("Al"),
        source="room-temperature conventional value 4.0495 A; the AFLOW Al1_ICSD_52611 primitive (2.856 A) con
    verts to 4.0398 A, a different entry"),
    "cu": Phase(
        key="cu", name="Copper", formula="Cu", space_group="Fm-3m (225)",
        a=3.6149, sites=_fcc("Cu"),
        source="room-temperature conventional value 3.6149 A; AFLOW Cu1_ICSD_53755 converts to 3.6146 A"),
    "fe": Phase(
        key="fe", name="alpha-Iron", formula="Fe", space_group="Im-3m (229)",
        a=2.8665, sites=(Site("Fe", 0, 0, 0), Site("Fe", .5, .5, .5)),
        source="room-temperature conventional value; the AFLOW ICSD entry reads 2.9647 A and is flagged as exp
    anded"),
    "nacl": Phase(
        key="nacl", name="Rock salt", formula="NaCl", space_group="Fm-3m (225)",
        a=5.6402, sites=_rocksalt("Na", "Cl"),
        source="room-temperature conventional value 5.6402 A; AFLOW Cl1Na1_ICSD_60280 converts to 5.665 A"),
    "mgo": Phase(
        key="mgo", name="Periclase", formula="MgO", space_group="Fm-3m (225)",
        a=4.2112, sites=_rocksalt("Mg", "O"),
        source="room-temperature conventional value 4.2112 A; AFLOW Mg1O1_ICSD_173128 converts to 4.253 A"),
    "zno": Phase(
        key="zno", name="Zincite", formula="ZnO", space_group="P63mc (186)",
        a=3.2494, c=5.2038, hexagonal=True, sites=(
            Site("Zn", 1/3, 2/3, 0.0), Site("Zn", 2/3, 1/3, 0.5),
            Site("O", 1/3, 2/3, 0.3830), Site("O", 2/3, 1/3, 0.8830)),
        source="COD 9004180, a = 3.2494, c = 5.2038 A; u(O) = 0.3830"),
    "tio2-rutile": Phase(
        key="tio2-rutile", name="Rutile", formula="TiO2", space_group="P42/mnm (136)",
        a=4.5937, c=2.9587, sites=(
            Site("Ti", 0, 0, 0), Site("Ti", .5, .5, .5),
            Site("O", .3047, .3047, 0), Site("O", .6953, .6953, 0),
            Site("O", .8047, .1953, .5), Site("O", .1953, .8047, .5)),
        source="room-temperature conventional values a = 4.5937, c = 2.9587 A; u(O) = 0.3047. COD 9004143 is a
     600 C record, which is why its cell reads 4.616/2.977 A"),
    "tio2-anatase": Phase(
        key="tio2-anatase", name="Anatase", formula="TiO2", space_group="I41/amd (141)",
        a=3.7845, c=9.5143, sites=(
            Site("Ti", 0, 0, 0), Site("Ti", 0, .5, .25), Site("Ti", .5, .5, .5), Site("Ti", .5, 0, .75),
            Site("O", 0, 0, .2079), Site("O", 0, .5, .4579), Site("O", .5, .5, .7079),
            Site("O", .5, 0, .9579), Site("O", 0, .5, .0421), Site("O", 0, 0, .7921),
            Site("O", .5, 0, .5421), Site("O", .5, .5, .2921)),
        source="room-temperature cell a = 3.7845, c = 9.5143 A; z(O) = 0.2079 with the basis expanded by gemmi
     from COD 9008214, a 300 C record"),
}


def get_phase(key: str) -> Phase:
    try:
        return PHASES[key]
    except KeyError:
        raise KeyError(f"unknown phase {key!r}; choose from {', '.join(sorted(PHASES))}") from None
```

xrdsim/sources.py

```
"""X-ray source: the copper tube and its emission lines.

Wavelengths in angstrom. The K-alpha doublet is what a powder pattern shows; K-beta is
normally removed by a filter or a monochromator and is only modelled to show why.
"""

from __future__ import annotations
from dataclasses import dataclass
```

```
CU_KA1 = 1.540598
CU_KA2 = 1.544426
CU_KB = 1.392250

# K-alpha2 / K-alpha1 intensity ratio, and K-beta / K-alpha ratio for a Cu tube.
KA2_TO_KA1 = 0.497
KB_TO_KA = 0.20


@dataclass(frozen=True)
class Line:
    wavelength: float
    weight: float


def emission_lines(ka2: bool = True, kbeta: bool = False) -> list[Line]:
    """Emission lines of a Cu tube with their relative intensities."""
    lines = [Line(CU_KA1, 1.0)]
    if ka2:
        lines.append(Line(CU_KA2, KA2_TO_KA1))
    if kbeta:
        lines.append(Line(CU_KB, KB_TO_KA * (1.0 + (KA2_TO_KA1 if ka2 else 0.0))))
    return lines


def weighted_wavelength(lines: list[Line]) -> float:
    """Intensity-weighted mean wavelength of the lines in use."""
    total = sum(line.weight for line in lines)
    if total <= 0:
        raise ValueError("no lines")
    return sum(line.wavelength * line.weight for line in lines) / total
```

xrdsim/structure.py

```
"""Structure factors, multiplicities and systematic absences.

A phase carries its cell and an explicit, symmetry-expanded basis; this module turns
that into |F(hkl)|^2 and the multiplicity the powder average needs.
"""

from __future__ import annotations

import cmath
import math

from .atoms import debye_waller, f0
from .lattice import d_spacing


def structure_factor(phase, h: int, k: int, l: int, wavelength: float) -> complex:
    """F(hkl) in electrons, including the Debye-Waller term of every site."""
    d = d_spacing(h, k, l, **phase.cell_kwargs())
    s = 1.0 / (2.0 * d)                 # sin(theta)/lambda, by Bragg's law
    total = 0j
    for site in phase.sites:
        phase_angle = 2.0 * math.pi * (h * site.x + k * site.y + l * site.z)
        total += (site.occupancy * f0(site.element, s) * debye_waller(site.b_iso, s)
                  * cmath.exp(1j * phase_angle))
    return total


def multiplicity(h: int, k: int, l: int, system: str) -> int:
    """Number of symmetry-equivalent planes in the powder average."""
    h, k, l = abs(h), abs(k), abs(l)
```

```
    if system == "cubic":
        perms = 6 if len({h, k, l}) == 3 else (3 if len({h, k, l}) == 2 else 1)
        return perms * 2 ** sum(v != 0 for v in (h, k, l))
    if system == "tetragonal":
        hk = 2 if h != k else 1
        return (hk * 2 ** sum(v != 0 for v in (h, k))) * (2 if l else 1)
    if system == "hexagonal":
        if h == 0 and k == 0:
            return 2 if l else 1
        # (h, k) on a mirror plane (h == 0, k == 0 or h == k) has a 6-member in-plane orbit;
        # a general (h, k) has 12. A non-zero l doubles either.
        in_plane = 6 if (h == 0 or k == 0 or h == k) else 12
        return 2 * in_plane if l else in_plane
    raise ValueError(f"multiplicity is not defined for {system!r}")


def lorentz_polarisation(two_theta_deg: float, monochromator_two_theta: float | None = None) -> float:
    """LP factor for an unpolarised tube, optionally with a monochromator."""
    t = math.radians(two_theta_deg)
    theta = t / 2.0
    cos2t = math.cos(t) ** 2
    if monochromator_two_theta is None:
        numerator = 1.0 + cos2t
    else:
        numerator = 1.0 + cos2t * math.cos(math.radians(monochromator_two_theta)) ** 2
    return numerator / (math.sin(theta) ** 2 * math.cos(theta))


def relative_intensity(phase, h: int, k: int, l: int, wavelength: float,
                       monochromator_two_theta: float | None = None) -> float:
    """I/I0 for one reflection: |F|^2 m LP, normalised later across the pattern."""
    f = structure_factor(phase, h, k, l, wavelength)
    d = d_spacing(h, k, l, **phase.cell_kwargs())
    two_theta = math.degrees(2.0 * math.asin(wavelength / (2.0 * d)))
    m = multiplicity(h, k, l, phase.system)
    return abs(f) ** 2 * m * lorentz_polarisation(two_theta, monochromator_two_theta)
```